

Lab 09: Calling convention

Lab 9: Calling convention

Objectives: Calling convention

After completing this lab, you will:

- Write RISC-V functions, pass parameters, and return results.
- Understand the stack segment, allocate, and free stack frames.
- Understand the RISC-V register usage convention.
- Write recursive functions in RISC-V.

Calling convention

According to the calling convention, when we call a function, that function promises to leave some, but not all, registers unchanged. The caller is the function making the call, and the callee is the function being called. All functions have the ability to be a caller and a callee; we, as programmers, should never assume a function isn't something because we have no knowledge about where a function may or may not be called from. Functions that call other functions are always a caller, even if they're by default a callee.

The registers that a function promises to leave unchanged are the callee-saved registers (preserved registers). `s0` through `s11` (saved registers) and `sp` are preserved registers. The registers that a function does not promise to leave unchanged are the caller-saved registers (non-preserved registers). `a0` through `a7` (argument registers), `t0` through `t6` (temporary registers), and `ra` (return address) are non-preserved registers.

The caller perspective

When we call a function, that function promises to not modify any of the preserved registers, from the perspective of the caller. This means that when the function returns, we can be sure that the preserved registers have not changed. This does not however, guarantee that function called will not modify preserved register values across the function call; it just guarantees that from the perspective of the caller, the preserved register values appear unchanged.

```
1 addi s0, x0, 5      # s0 contains 5
2 jal ra, func        # call a function
3 addi s0, s0, 0      # s0 should contain 5 here!
```

Listing 1: Preserving registers after function call

However, that function is allowed to freely modify any of the non-preserved registers. This means that as soon as you call a function and the function returns, every non-preserved register now contains garbage. You do not know what values are in the non-preserved registers anymore.

NOTE: garbage refers to unknown values; even if the values in non-preserved registers remain unchanged across a function call. This is because our guarantees don't say that non-preserved registers are untouched, so even if they were not changed, we have to assume they were.

```
1 addi t0, x0, 5      # t0 contains 5
2 jal ra, func        # call a function
3 addi t0, t0, 0      # t0 contains garbage!
```

Listing 2: Non-preserved registers after function call

This is a common calling convention bug: when a function returns, you cannot rely on the values in any non-preserved register. One way to avoid this bug is to save the values in the non-preserved registers on the stack before calling the function, then restore the values in the non-preserved registers after the function returns. For example:

```
1 addi t0, x0, 5      # t0 contains 5
2 addi a0, t0, 10     # a0 (argument to func) contains 10
3
4 # Prologue
5 addi sp, sp, -8     # decrement stack pointer
6 sw t0, 0(sp)        # store t0 value on the stack
7 sw a0, 0(sp)        # store a0 value on the stack
8
9 # Function call
10 jal ra, func        # call function
11 mv s0, a0           # save return value 1, before restoring a0's old value
12                    # to avoid overwriting the return value.
13 mv s1, a1           # save return value 2, before moving on,
14                    # in case a1 is used in the future, to avoid
15                    # potentially overwriting the return value.
16
17 # Epilogue
18 lw t0, 0(sp)        # restore t0 value from the stack
19 lw a0, 4(sp)        # restore a0 value from the stack
20 addi sp, sp, 8      # increment stack pointer
21
22 addi t0, t0, 0      # t0 contains 5 here because we saved it before
23                    # the function call
24 xori t1, a0, t0
```

Listing 3: Saving register in the stack before calling

The callee perspective

When we write a function, we are allowed to freely change any of the non-preserved registers. However, we must promise not to change any of the preserved registers from the perspective of the caller.

This is another common calling convention bug: a function cannot noticeably change any preserved registers. In other words, change whatever you'd like, callee, just make sure you restore the preserved register values before returning back to the caller and don't worry about the non-preserved registers.

If we want to use one of the preserved registers during the function, we need to save the register values on the stack at the start of the function, and then restore the values at the end of the function. For example:

```
1 # Prologue
2 addi sp, sp, -12    # decrement stack pointer
```

```

3  sw ra, 0(sp)           # store ra value on the stack
4  sw s0, 4(sp)           # store s0 value on the stack
5  sw s1, 0(sp)           # store s1 value on the stack
6
7  # do stuff in the function
8
9  # Epilogue
10 lw ra, 0(sp)           # restore ra value from the stack
11 lw s0, 4(sp)           # restore s0 value from the stack
12 lw s1, 0(sp)           # restore s1 value from the stack
13 addi sp, sp, 12         # increment stack pointer
14
15 ret                     # return from function

```

Listing 4: Saving register in the stack after calling

Notice that we also saved the value of `ra` on the stack. Remember that the `ra` register stores the address that we'll jump back to after this function finishes executing. Saving the value of `ra` on the stack is necessary if we decide to call another function inside this function. If we make a function call at the "do stuff" comment, then that function call will overwrite `ra`, and we'll lose the address that we were supposed to jump back to.

Coding advice

When following calling convention, it's better to be safe than sorry! It doesn't hurt to save an extra register you didn't need to save, but it's very bad to forget to save a register that you were supposed to save. With that being said, don't save every register because it takes up more stack space than necessary and also muddles your code, because eventually it's unclear as to what registers are being actively used.

1. Always save the value of `ra` at the start of a function and restore it at the end of a function. Even though it looks like it's being saved in the callee-prologue and restored in the callee-epilogue as a caller-saved register, it's actually preemptively saving `ra` in case at any point a function is called, and guarantees the return address we want to return to is preserved, before doing anything else.
2. Save the values of all the preserved registers at the start of a function and restore them at the end of the function in the prologue and epilogue. If you choose not to save the value in one of the save registers, be absolutely sure that you aren't using that register in the function.
3. Save the values of the non-preserved registers that we rely on after a function is being called; in other words, only non-preserved registers values we rely on across a function call should be saved. Otherwise, they can be discarded. To do this, save them before calling a function and restore them after calling a function. You can do this by moving the temporary values by moving them to a free saved register, if any are available, or by saving them on the stack. If you choose not to save the value in one of the non-preserved registers, be absolutely sure that you do not rely on the value in that register after the function returns.

Lab Assignment

1. Write a function `exp` in RISC-V that, when given a positive integer x , returns $x^5 + 6x^3 + 3x + 4$. Your code should include the following:
 - Write a main function to call the `exp` function and print the output in the console.
 - Function `exp` will compute and return the expression. Function `exp` must call the following additional functions:
 - a. Call a power function to calculate x^n . You may assume n will be positive.
 - b. Call a times function to calculate $n \times x$.
2. **Challenge:** (you have one week to do it) Quicksort is a divide and conquer algorithm. Quicksort first divides a large array into two smaller sub-arrays: the low elements and the

high elements. Quicksort can then recursively:

- a. Pick an element, called a pivot, from the array.
- b. Reorder the array so that all elements with values less than the pivot come before the pivot, while all elements with values greater than the pivot come after it (equal values can go either way).

```
1: function PARTITION(A, lo, hi)
2: pivot = A[hi]
3: i = lo-1;
4: for (j = lo; j ≤ hi-1; j=j+1) do
5: if A[j] ≤ pivot then
6: i = i+1;
7: swap A[i] with A[j];
8: end if
9: end for
10: swap A[i+1] with A[hi];
11: return i+1;
12: end function
```

- c. Recursively apply the array partition to the sub-array of elements with smaller values and separately to elements with greater values.

```
1: function QUICKSORT(A, lo, hi)
2: if lo < hi then
3: p=partition(A, lo, hi);
4: quicksort(A, lo, p - 1);
5: quicksort(A, p + 1, hi);
6: end if
7: end function
```

- d. Implement Quicksort w.r.t. the following array [-1 22 8 35 5 4 11 2 1 78] in ascending order.